

## 0.1 Kiến trúc hệ thống

### 0.1.1 Nhận xét ban đầu về RL

Qua các thử nghiệm ban đầu, ta nhận thấy rằng việc sử dụng RL mà không cung cấp trước quỹ đạo chuyển động là một bài toán rất khó.

Cụ thể: Agent phải học đồng thời: lập kế hoạch đường đi; điều chỉnh vận tốc; bù sai lệch của mô hình lằng lộn; không gian hành động và trạng thái quá lớn, khả năng hội tụ thấp.

Việc không hội tụ này là một kết quả đáng ghi nhận. Bây giờ ta chuyển trọng tâm sang xây dựng baseline dựa trên lập kế hoạch truyền thống (Slicing).

## 0.2 Kiến trúc đề xuất: Slicer vs. Agent

Thay vì để RL điều khiển toàn bộ hệ thống, ta tách thành hai phần: Slicing (lập kế hoạch truyền thống) và Printing (in + hiệu chỉnh bằng RL)

## 0.3 Đầu vào hệ thống

Hệ thống nhận các đầu vào sau:

- Bản đồ chiều cao mục tiêu:  $h_{\text{target}}(x, y)$
- Miền tham số:  $v_{\text{max}}$ , feed rate, v.v.

Trong trường hợp 1D, ta có:  $h_{\text{target}}(x)$  với  $x \in [0, x_{\text{max}}]$

## 0.4 Bước 1: Slicing (Chuẩn bị)

### 0.4.1 Chia hình học theo layer

Cho trước một độ dày layer  $\Delta h$ . Ta chia hình học thành các lớp:

$$L_k = \{x \mid h_{\text{target}}(x) \in [k\Delta h, (k+1)\Delta h]\}$$

Mỗi lớp có một mục tiêu chiều cao riêng:  $h_{\text{target}}^{(k)}(x)$

### 0.4.2 Tính toán vận tốc cơ sở

Giả sử ta có chiều cao ban đầu:  $h_0(x)$ . Ta cần in thêm:

$$\Delta h(x) = h_{\text{target}}^{(k)}(x) - h_0(x)$$

Giả sử mô hình lắng đọng là tích chập:

$$h(x) = \int K(x - \xi) m(\xi) d\xi$$

với  $K$  là kernel lắng đọng,  $m(\xi)$  là lượng vật liệu phun

Vì  $m(\xi) = \frac{f}{v(\xi)}$  nên bài toán tìm vận tốc cơ sở  $v_{\text{base}}(x)$  trở thành bài toán tối ưu:

$$\min_{v(x)} \left\| h_0(x) + \text{Deposition}(v(x)) - h_{\text{target}}^{(k)}(x) \right\|$$

### 0.4.3 Đầu ra của Slicer

Slicer trả về quỹ đạo vòi phun:  $\{(x_i, y_i), v_i^{\text{base}}\}_{i=1}^S$  với các điểm cách nhau một khoảng cố định  $ds$ ,  $v_i^{\text{base}}$  là vận tốc cơ sở tại điểm  $i$ . Nếu mô hình lắng đọng là hoàn hảo, việc in với  $v^{\text{base}}$  sẽ đạt đúng mục tiêu.

## 0.5 Bước 2: Printing (In vs. RL hiệu chỉnh)

Vòi phun di chuyển theo quỹ đạo đã được tính toán, agent chỉ thực hiện hiệu chỉnh nhỏ:  $v_i = v_i^{\text{base}} + \delta v_i$  hoặc có thể thêm lệch ngang nhỏ.

### 0.5.1 Vai trò của RL

Nếu mô hình lắng đọng hoàn toàn chính xác, thì:  $\delta v_i = 0$ . Agent không cần làm gì. Trong thực tế, tồn tại sai lệch giữa: mô hình mô phỏng (simulation) và hệ thống thật. Ta có thể mô hình hóa sai lệch bằng thay đổi kernel hoặc thêm nhiễu ngẫu nhiên. Agent học cách bù sai lệch này.

## 0.6 Điều kiện để mở rộng sang Multilayer

Để in nhiều layer ổn định, cần agent đạt được nhiều dạng  $h_{\text{target}}$  khác nhau và xử lý được nhiều nền ban đầu  $h_0(x)$  khác nhau. Điều này có thể triển khai trong 1D bằng cách random hóa target và  $h_0(x)$ .

## 0.7 Cấu trúc luận văn đề xuất

1. Động cơ nghiên cứu
2. Reinforcement Learning (REINFORCE  $\rightarrow$  PPO)
3. Mô hình hóa Cold Spray Additive Manufacturing
  - Mô hình lắng đọng
  - Sai lệch so với thực tế
  - Slicing / Path Planning
  - Thiết kế Agent và Reward
4. Thực nghiệm
  - 1D với target biến thiên
  - 1D với target + nền biến thiên
  - Multi-layer: kết hợp Slicer và Agent
  - 2D

## 0.8 Ý tưởng tổng thể

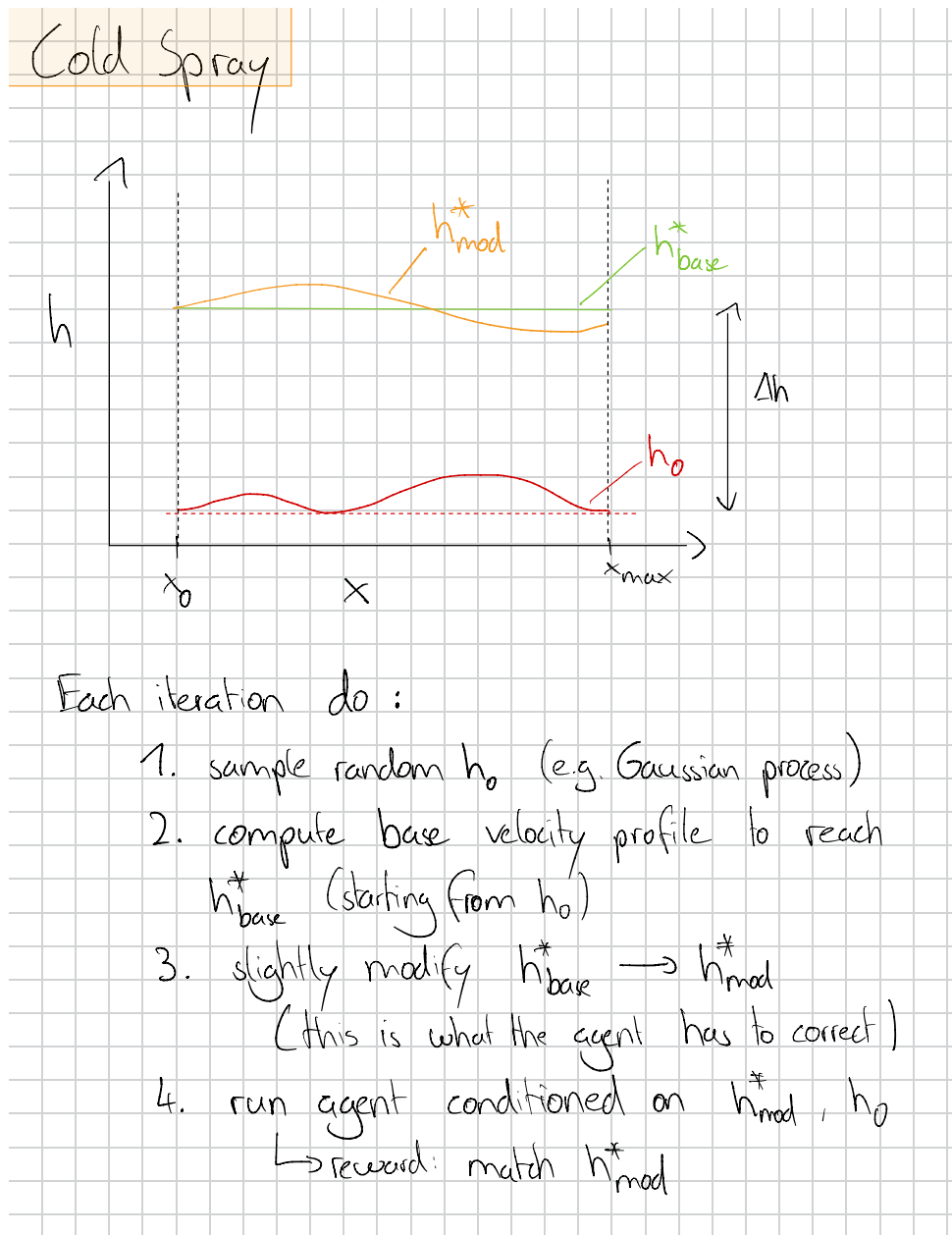
Workflow cuối cùng:

Slicer  $\longrightarrow$  Agent hiệu chỉnh  $\longrightarrow$  Multi-layer printing

Phần thực nghiệm được chia thành nhiều bước trung gian, dần dần tiến đến hệ thống hoàn chỉnh.

## 0.9 Giải thích hình minh họa quy trình Slicer vs. Agent (1D)

Hình minh họa mô tả bài toán in một layer trong Cold Spray dưới dạng 1D.



### 0.9.1 Thiết lập hình học

Trục ngang biểu diễn vị trí không gian  $x \in [x_0, x_{\max}]$ ; trục đứng biểu diễn chiều cao vật liệu  $h(x)$

Tại mỗi lần in (mỗi iteration), ta xét 3 đường cong chính: đường màu đỏ  $h_0(x)$ ; đường màu xanh lá  $h_{\text{base}}^*(x)$ ; đường màu cam  $h_{\text{mod}}^*(x)$

### 0.9.2 Nền ban đầu (Boden)

Đường màu đỏ  $h_0(x)$  là chiều cao hiện tại trước khi in layer mới. Trong mô hình nhiều lớp (multi-layer),  $h_0(x)$  chính là kết quả của các lớp trước đó. Trong thí nghiệm 1D, ta có thể:

- Đặt  $h_0(x) = 0$  (trường hợp đơn giản),
- Hoặc lấy  $h_0(x)$  ngẫu nhiên (ví dụ từ Gaussian Process).

Việc cho  $h_0(x)$  ngẫu nhiên giúp agent học cách xử lý các điều kiện nền khác nhau.

### 0.9.3 Mục tiêu lý tưởng của Slicer

Đường màu xanh  $h_{\text{base}}^*(x)$  là chiều cao mục tiêu lý tưởng sau khi in layer, được tính toán bởi Slicer dưới giả thiết mô hình lắng đọng hoàn toàn chính xác, không có nhiễu, không có sai lệch giữa mô phỏng và thực tế. Slicer sẽ giải bài toán tối ưu:

$$\min_{v_{\text{base}}(x)} \|h_0(x) + \text{Deposition}(v_{\text{base}}(x)) - h_{\text{base}}^*(x)\|$$

Kết quả của Slicer là một quỹ đạo vòi phun:  $\{(x_i, v_i^{\text{base}})\}_{i=1}^S$  trong đó:

- $x_i$  cách nhau một khoảng cố định  $ds$ ,
- $v_i^{\text{base}}$  là vận tốc cơ sở.

### 0.9.4 Sai lệch thực tế

Đường màu cam  $h_{\text{mod}}^*(x)$  là mục tiêu thực tế khi xét đến sai lệch giữa mô hình và hệ thống thật. Sai lệch có thể được mô hình hóa bằng thay đổi nhẹ kernel lắng đọng, thêm nhiễu ngẫu nhiên, sai số trong feed rate. Do đó:  $h_{\text{mod}}^*(x) \neq h_{\text{base}}^*(x)$ . Agent cần bù phần sai lệch này.

### 0.9.5 Lượng vật liệu cần bù

Hiệu độ cao cần in thêm là:  $\Delta h(x) = h_{\text{target}}(x) - h_0(x)$  trong đó  $h_{\text{target}}(x) = h_{\text{mod}}^*(x)$

### 0.9.6 Quy trình mỗi iteration

Tại mỗi iteration (theo hình):

1. Sinh ngẫu nhiên nền:  $h_0(x)$
2. Tính vận tốc cơ sở để đạt:  $h_{\text{base}}^*(x)$
3. Chỉnh sửa nhẹ mục tiêu:  $h_{\text{base}}^*(x) \rightarrow h_{\text{mod}}^*(x)$
4. Chạy agent với đầu vào:  $(h_0(x), h_{\text{mod}}^*(x))$  và hành động:  $v_i = v_i^{\text{base}} + \delta v_i$

Hàm thưởng được thiết kế để tối thiểu hóa sai số:  $R = -\|h_{\text{final}}(x) - h_{\text{mod}}^*(x)\|$

### 0.9.7 Lộ trình mở rộng

Để hệ thống hoạt động cho nhiều layer:

- Cần random hóa  $h_0(x)$ ,
- Cần random hóa  $h_{\text{target}}(x)$ ,
- Sau đó kết hợp cả hai.

Chiến lược thực nghiệm:

1. Chỉ random target.
2. Chỉ random nền.
3. Random cả hai.